

Web Application Security Testing Report

1. Introduction

The objective of this task was to perform practical web application security testing on the Damn Vulnerable Web Application (DVWA) in a controlled local laboratory environment. The testing focused on identifying common web vulnerabilities, understanding their exploitation techniques, analyzing HTTP requests and responses using Burp Suite, and implementing basic security mitigations.

The following areas were tested:

- SQL Injection
- Cross-Site Scripting (XSS)
- Cross-Site Request Forgery (CSRF)
- Local File Inclusion (LFI)
- Remote File Inclusion (RFI) configuration testing
- Burp Suite request interception and modification
- Burp Suite Intruder fuzzing
- HTTP Security Headers

All testing was performed against the locally hosted DVWA instance and was conducted for educational and authorized security-testing purposes.

2. Objectives

The main objectives of Task 3 were:

1. To understand and demonstrate common web application vulnerabilities.
2. To identify how insecure input handling can lead to security issues.
3. To analyze HTTP requests and responses using Burp Suite.
4. To modify HTTP requests and observe their effect on application behavior.
5. To perform basic fuzzing using Burp Suite Intruder.
6. To analyze HTTP security headers.
7. To configure appropriate security headers in Apache.
8. To understand mitigation techniques for the identified vulnerabilities.
9. To document the testing process with screenshots and observations.

3. Testing Environment

Component	Configuration
Operating System	Kali Linux
Web Application	Damn Vulnerable Web Application (DVWA)
Web Server	Apache 2.4.68
Backend	PHP / MySQL
Browser	Firefox / Chromium
Security Testing Tool	Burp Suite Community Edition
Target	http://localhost/dvwa/
Security Level	Low for vulnerability demonstrations
Testing Type	Local / Controlled Web Application Security Testing

The DVWA application was hosted locally, ensuring that all vulnerability testing was performed within an isolated and controlled environment.

4. SQL Injection

4.1 Objective

The objective of this test was to determine whether user-controlled input could be manipulated to alter the SQL query executed by the application.

4.2 Vulnerability Description

SQL Injection occurs when an application incorporates user-supplied input directly into an SQL query without properly validating or parameterizing the input.

An attacker may manipulate the application's input fields to change the intended SQL query and potentially access, modify, or retrieve unauthorized database information.

4.3 Testing Procedure

1. Opened the DVWA application.
2. Navigated to the **SQL Injection** module.
3. Ensured that the DVWA security level was set to **Low**.
4. Entered a normal user identifier to observe the application's normal behavior.
5. Tested the input with SQL manipulation characters/payloads.
6. Observed the resulting application response.
7. Compared the normal and manipulated responses.

4.4 Observation

The application processed user input directly through the vulnerable SQL Injection functionality.

The behavior demonstrated that insufficient input validation can allow SQL syntax to influence the application's database query.

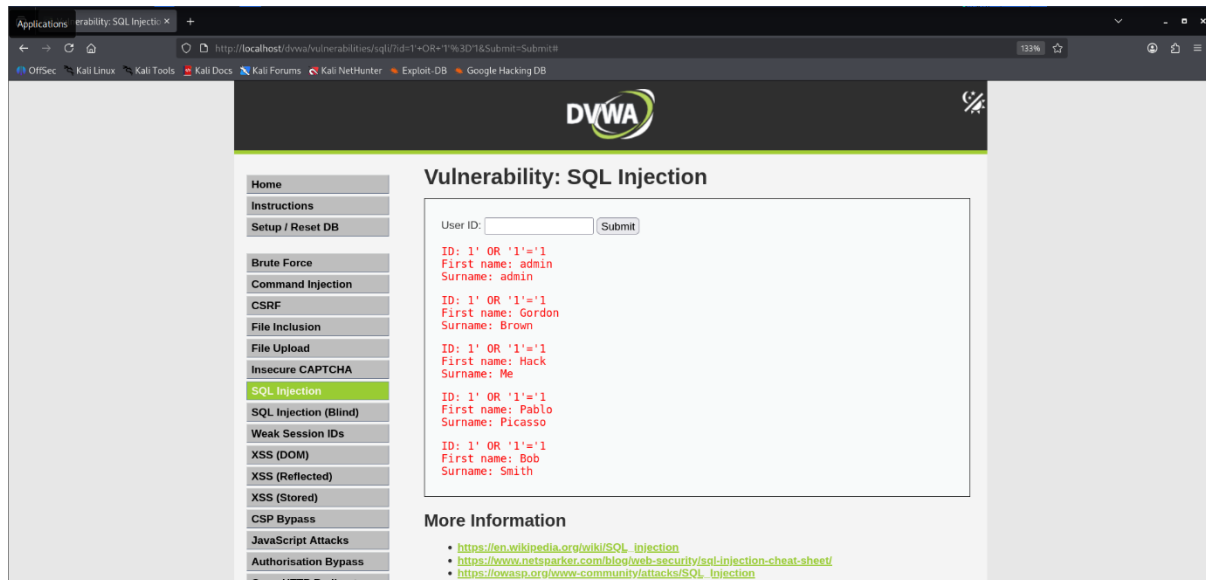


Figure 1: SQL Injection testing in DVWA

4.5 Impact

A successful SQL Injection vulnerability can potentially allow an attacker to:

- Retrieve unauthorized database information.
- Bypass application authentication.
- Enumerate database records.
- Modify or delete database information.
- Potentially compromise other application functionality depending on database privileges.

4.6 Mitigation

SQL Injection can be mitigated by:

- Using prepared statements and parameterized queries.
- Avoiding direct concatenation of user input into SQL queries.
- Implementing server-side input validation.
- Applying the principle of least privilege to database accounts.
- Avoiding detailed database error messages in production environments.

5. Cross-Site Scripting (XSS)

5.1 Objective

The objective was to determine whether user-supplied input could be interpreted as executable JavaScript by the browser.

5.2 Vulnerability Description

Cross-Site Scripting (XSS) occurs when an application includes untrusted user input in a web page without appropriate output encoding or sanitization.

Depending on the type of XSS, malicious scripts may execute in another user's browser within the security context of the vulnerable website.

5.3 Testing Procedure

Reflected XSS:

1. Opened DVWA and navigated to XSS (Reflected).
2. Set the security level to Low.
3. Entered normal input to observe the application's behavior.
4. Tested HTML/JavaScript input in the vulnerable field.
5. Observed whether the supplied input was reflected in the response and interpreted by the browser.

Stored XSS:

1. Navigated to **XSS (Stored)**.
2. Set the security level to **Low**.
3. Entered normal input to observe the application's behavior.
4. Submitted a JavaScript payload through the vulnerable input field.
5. Reloaded/accessed the relevant page to determine whether the input had been stored and subsequently interpreted by the browser.

5.4 Observation

Two forms of XSS were tested in DVWA:

- **Reflected XSS:** User-supplied input was reflected by the application and could be interpreted as HTML/JavaScript by the browser.
- **Stored XSS:** User-supplied input was stored by the application and subsequently interpreted when the affected content was accessed.

Screenshots:

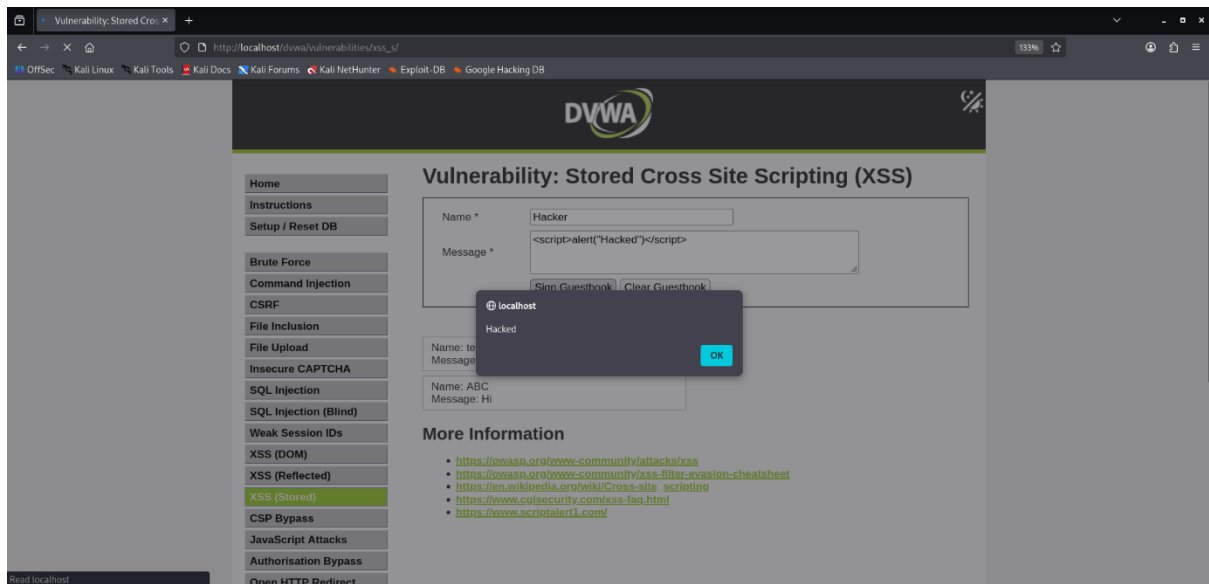


Figure 2: Reflected XSS demonstration in DVWA

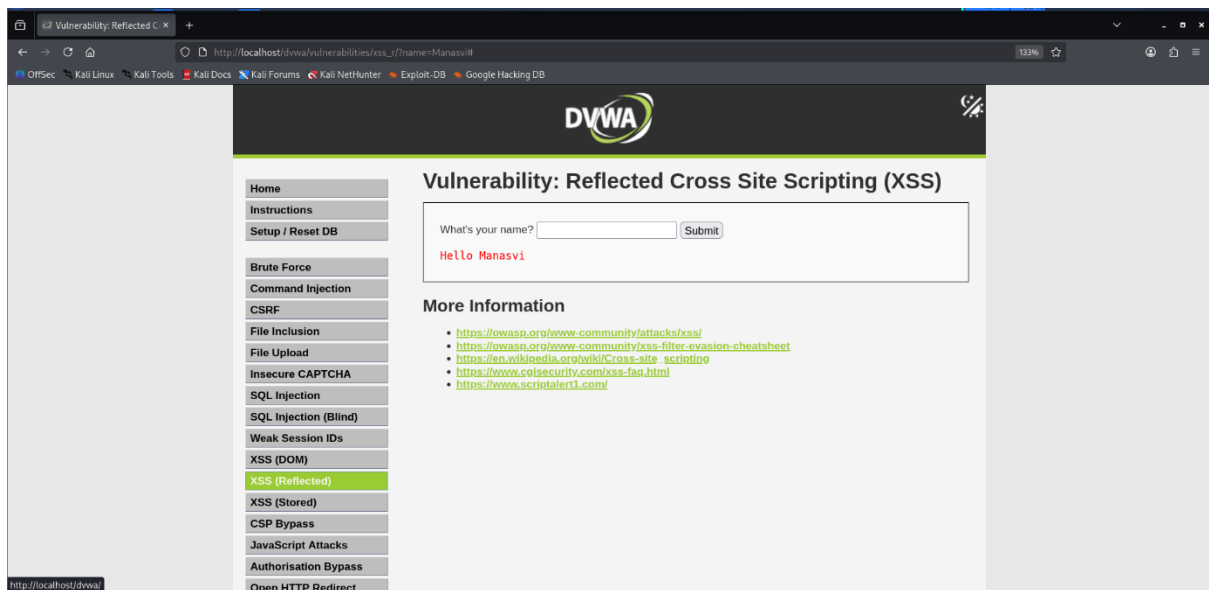


Figure 3: Stored XSS demonstration in DVWA

5.5 Impact

XSS may allow an attacker to:

- Execute JavaScript in a victim's browser.
- Manipulate page content.
- Perform actions on behalf of the victim.
- Steal sensitive information accessible to client-side scripts.
- Conduct phishing or session-related attacks depending on application protections.

5.6 Mitigation

Recommended mitigations include:

- Context-aware output encoding.
- Input validation.
- Avoiding unsafe HTML rendering.
- Implementing an appropriate Content Security Policy (CSP).
- Using secure cookie attributes such as `HttpOnly` and `SameSite`.

6. Cross-Site Request Forgery (CSRF)

6.1 Objective

The objective was to understand whether a state-changing request could be submitted without adequate verification that the request originated from the legitimate application interface.

6.2 Vulnerability Description

Cross-Site Request Forgery (CSRF) occurs when an application accepts state-changing requests without sufficiently verifying that they were intentionally initiated by the authenticated user.

6.3 Testing Procedure

1. Opened the DVWA **CSRF** module.
2. Set the security level to **Low** for the vulnerability demonstration.
3. Examined the password-change functionality.
4. Captured/observed the relevant HTTP request.
5. Tested modification/replay of the request.
6. The application was subsequently tested with CSRF protection enabled/higher security settings.

6.4 Observation

During testing, the application displayed:

"CSRF token is incorrect"

when the request was submitted without the correct CSRF token.

This indicates that the application was performing token validation for the tested request.

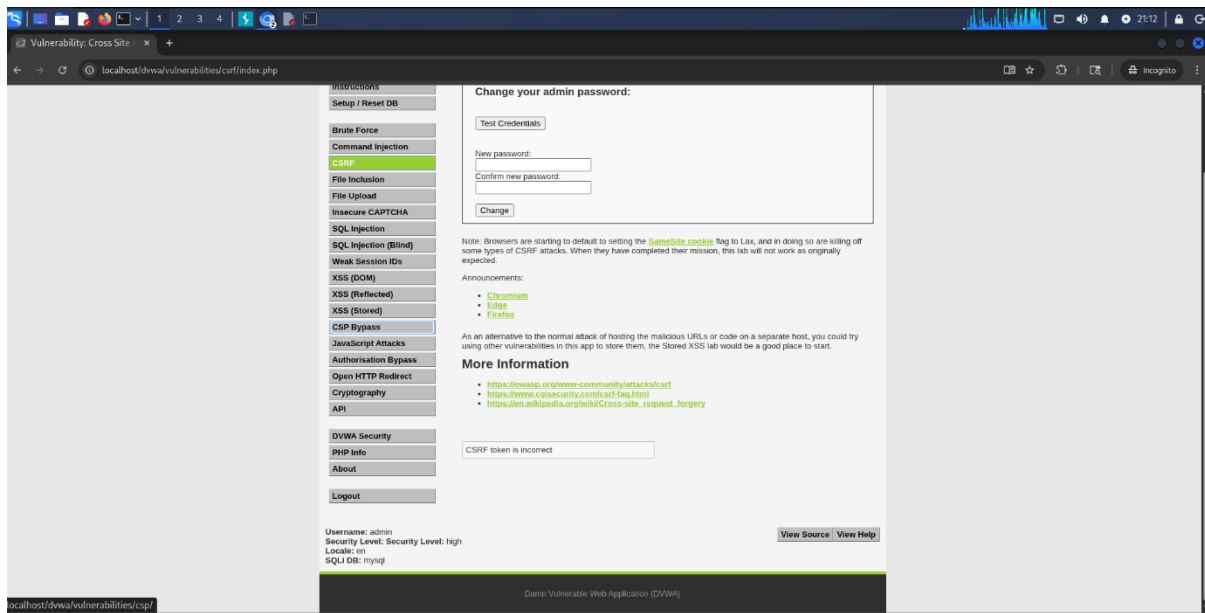


Figure 4: DVWA rejecting a request because of an incorrect CSRF token

6.5 Security Analysis

The CSRF token acts as an additional verification value that an attacker cannot simply assume or reuse arbitrarily.

The observed rejection demonstrates the effectiveness of CSRF-token validation in preventing unauthorized state-changing requests when the required token is missing or incorrect.

6.6 Mitigation

Recommended CSRF protections include:

- Using unpredictable CSRF tokens.
- Validating the token on every relevant state-changing request.
- Using SameSite cookie protections.
- Avoiding state-changing operations through GET requests.
- Implementing appropriate Origin/Referer validation where suitable.

7. Local File Inclusion (LFI)

7.1 Objective

The objective was to determine whether the DVWA File Inclusion functionality could be manipulated to access files outside the intended application resources.

7.2 Vulnerability Description

Local File Inclusion occurs when an application dynamically includes files based on user-controlled input without adequately restricting the permitted file paths.

If vulnerable, an attacker may be able to access sensitive files available on the local server.

7.3 Testing Procedure

1. Opened the DVWA **File Inclusion** module.
2. Set the security level to **Low**.
3. Observed the `page` parameter in the URL.
4. Tested the parameter with a local system file path.
5. The application successfully displayed the contents of the requested system file.

For example, the tested URL used the `page` parameter to reference the Linux password-file location.

7.4 Observation

The contents of `/etc/passwd` were displayed directly in the browser.

The output contained entries such as:

```
root:x:0:0:root:/root:/bin/bash
```

This demonstrated that the application was allowing a user-controlled parameter to reference a local file.

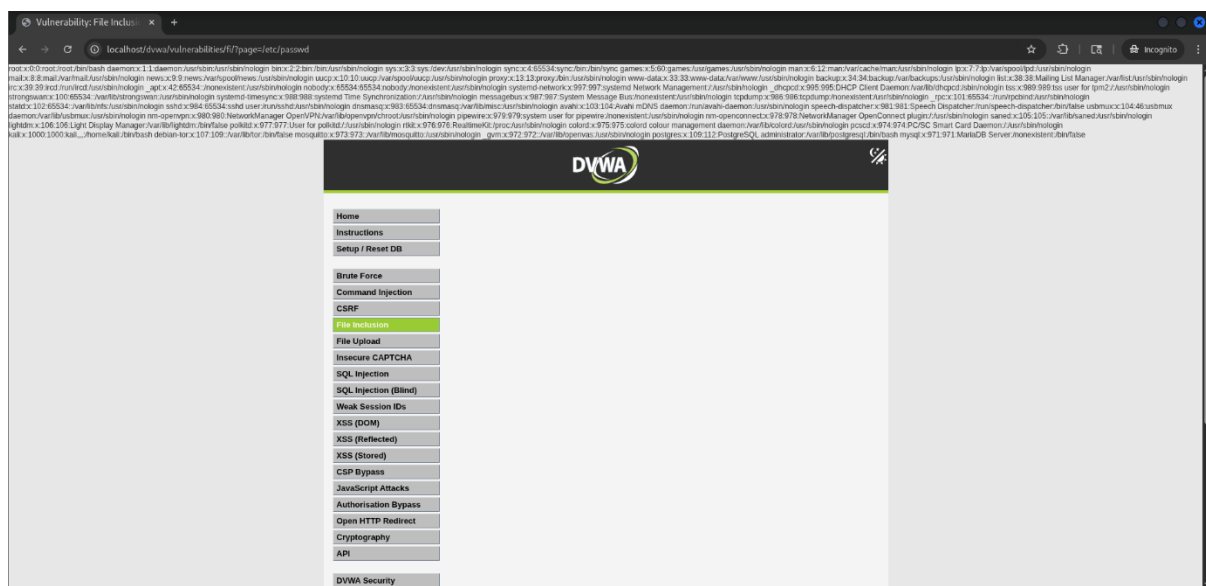


Figure 5: Local File Inclusion displaying /etc/passwd

7.5 Impact

LFI can potentially allow an attacker to:

- Read sensitive local files.
- Discover usernames and system information.
- Access application configuration files.
- Obtain credentials or secrets stored in files.
- Potentially escalate the attack when combined with other vulnerabilities.

7.6 Mitigation

LFI can be mitigated by:

- Avoiding direct use of user input in file inclusion operations.
- Using an allowlist of permitted files.
- Mapping user-controlled identifiers to predefined filenames.
- Validating and canonicalizing file paths.
- Restricting filesystem permissions.
- Disabling unnecessary file inclusion functionality.

8. Remote File Inclusion (RFI)

8.1 Objective

The objective was to determine whether the PHP/DVWA configuration permitted remote files to be included and potentially executed through the vulnerable `page` parameter.

8.2 Vulnerability Description

Remote File Inclusion occurs when an application allows an attacker-controlled remote resource to be included by the server.

If remote PHP inclusion is permitted, this can potentially result in execution of attacker-controlled PHP code.

8.3 Configuration Analysis

The PHP configuration was inspected before attempting RFI.

The relevant configuration values were:

```
allow_url_fopen = On
allow_url_include = Off
```

The important setting for PHP remote file inclusion is:

```
allow_url_include
```

Since it was configured as:

```
allow_url_include = Off
```

remote PHP file inclusion was disabled.

8.4 Testing Procedure

A remote URL was supplied to the `page` parameter as part of the RFI test.

The application returned:

```
ERROR: File not found!
```

The test was also performed using a local test server URL.

8.5 Observation

Remote file inclusion/execution could **not be successfully demonstrated** in the current PHP configuration.

This result is consistent with the configuration:

```
allow_url_include = Off
```

Core		
PHP Version	8.4.23	
Directive	Local Value	Master Value
<code>allow_url_fopen</code>	On	On
<code>allow_url_include</code>	Off	Off

Figure 6: PHP configuration showing `allow_url_include` disabled



Figure 7: RFI attempt returning "ERROR: File not found!"

8.6 Security Analysis

The failure of the RFI test does not indicate that the testing was unsuccessful. Instead, it demonstrates that the current PHP configuration prevents remote URL inclusion through the relevant PHP mechanism.

Therefore:

LFI was demonstrated successfully, while RFI was tested but could not be demonstrated because remote URL inclusion was disabled.

8.7 Mitigation

Recommended protections include:

- Keep `allow_url_include` disabled.
- Avoid dynamic file inclusion based directly on user input.
- Use allowlists for permitted resources.
- Apply strict server-side input validation.
- Restrict filesystem and application permissions.

9. Burp Suite — HTTP Request Interception and Modification

9.1 Objective

The objective was to understand how Burp Suite can intercept HTTP requests and modify parameters before forwarding them to the web application.

9.2 Procedure

1. Configured the browser to use Burp Suite as the HTTP proxy.
2. Opened the DVWA login functionality.
3. Enabled request interception in Burp Suite.
4. Submitted login credentials through the application.
5. Burp Suite intercepted the HTTP request.
6. The username parameter was modified in the intercepted request.
7. The modified request was forwarded to DVWA.
8. The application returned a login failure because the modified credentials were invalid.

9.3 Observation

The intercepted request could be modified before being sent to the server.

After changing the username parameter, DVWA returned a login failure.

This demonstrated that HTTP requests transmitted by a web application can be inspected and modified during security testing.

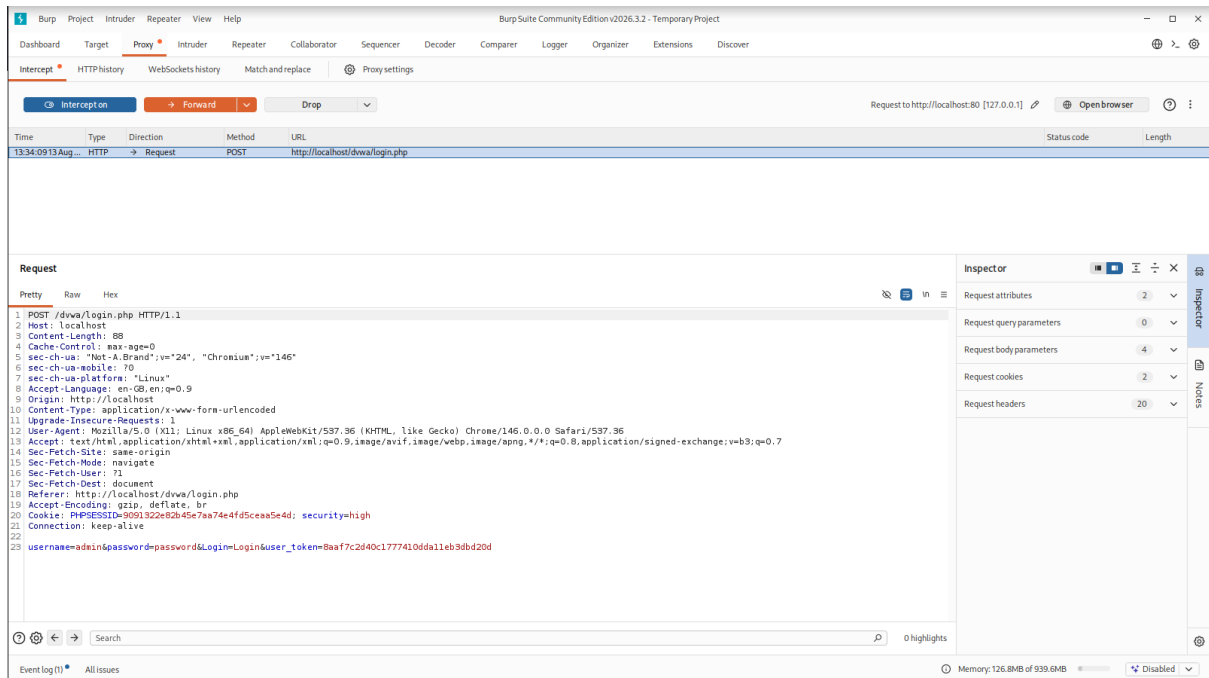


Figure 8: Intercepting and modifying a DVWA login request using Burp Suite

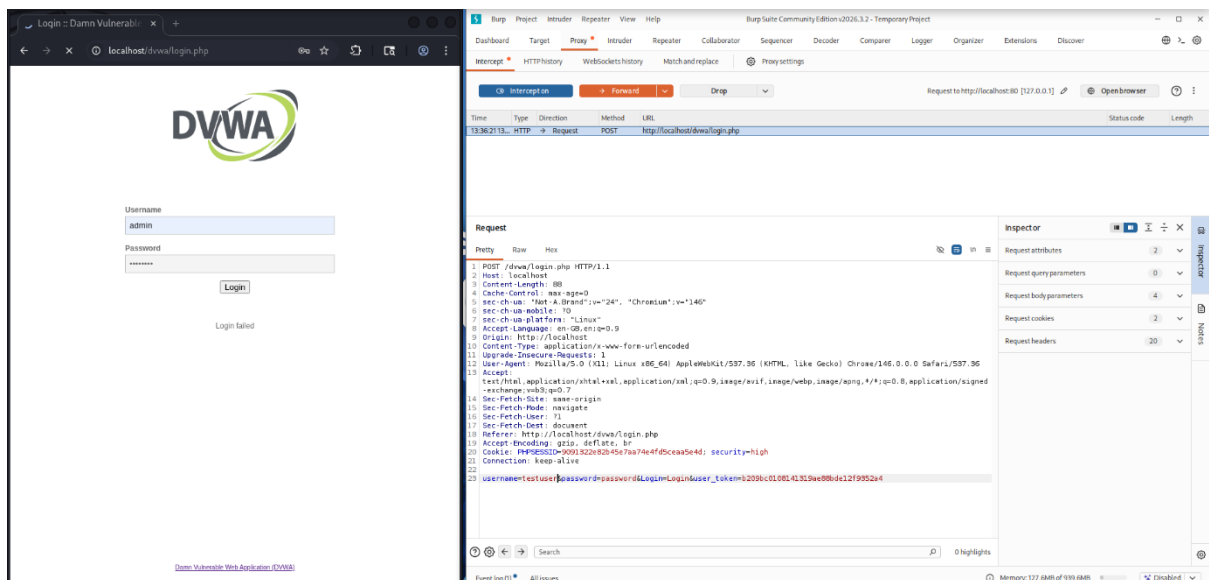


Figure 9: DVWA response after the modified login request

9.4 Security Significance

The test demonstrates why web applications must not rely on client-side controls or assume that HTTP parameters remain unchanged.

All security-sensitive validation must be performed on the server.

10. Burp Suite Intruder — Fuzzing

10.1 Objective

The objective was to use Burp Suite Intruder to automate the submission of multiple candidate values to a selected parameter and compare the resulting responses.

10.2 Procedure

1. Captured a DVWA login request using Burp Suite.
2. Sent the request to **Intruder**.
3. Selected the password parameter as the attack position.
4. Loaded a small test payload list.
5. Started the Intruder attack.
6. Compared the resulting HTTP responses.
7. Examined status codes, response lengths, and response contents.

10.3 Payloads Used

A small test payload list was used, including values such as:

```
admin
1234
test123
password
letmein
qwerty
```

10.4 Observation

The Intruder results showed multiple requests being generated with different payloads.

The results initially displayed different response characteristics depending on the DVWA security configuration.

After changing the DVWA security level to **Low**, the responses could be examined for meaningful differences.

The response corresponding to the valid credential combination contained:

"Welcome to the password protected area admin"

while unsuccessful attempts returned:

"Username and/or password incorrect"

This demonstrated how response content can be used as an indicator when analyzing automated requests.

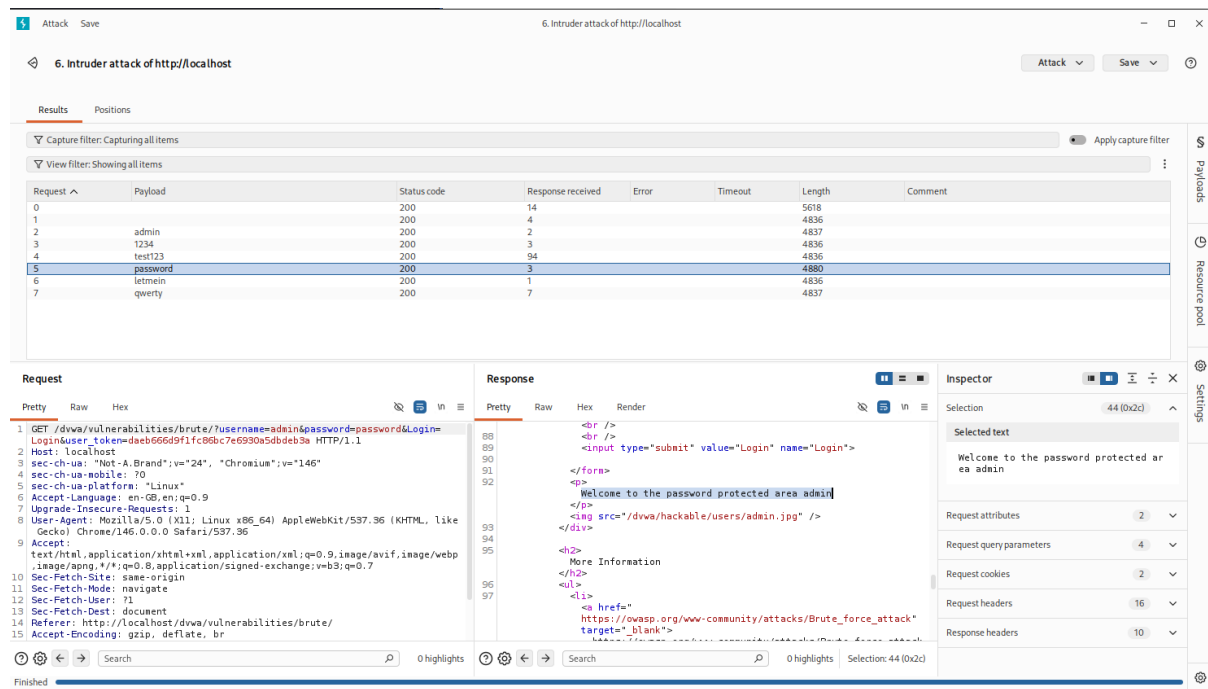


Figure 10: Burp Suite Intruder results with multiple test payloads

10.5 Security Significance

Intruder can help security testers identify differences in application responses during authorized testing.

Response status codes, lengths, content, and other characteristics can be used to identify anomalous or potentially successful requests.

10.6 Mitigation

Applications should implement:

- Strong password policies.
- Account lockout or temporary blocking.
- Rate limiting.
- Multi-factor authentication.
- CAPTCHA where appropriate.
- Monitoring and alerting for repeated authentication failures.

11. Web Security Headers

11.1 Objective

The objective was to analyze HTTP security headers and configure appropriate headers in Apache to improve the security posture of the web application.

11.2 Security Headers Analysis

The public test site `example.com` was analyzed using **Security Headers**.

The scanner produced an **F grade** and identified missing security headers including:

- Strict-Transport-Security
- Content-Security-Policy
- X-Frame-Options
- X-Content-Type-Options
- Referrer-Policy
- Permissions-Policy

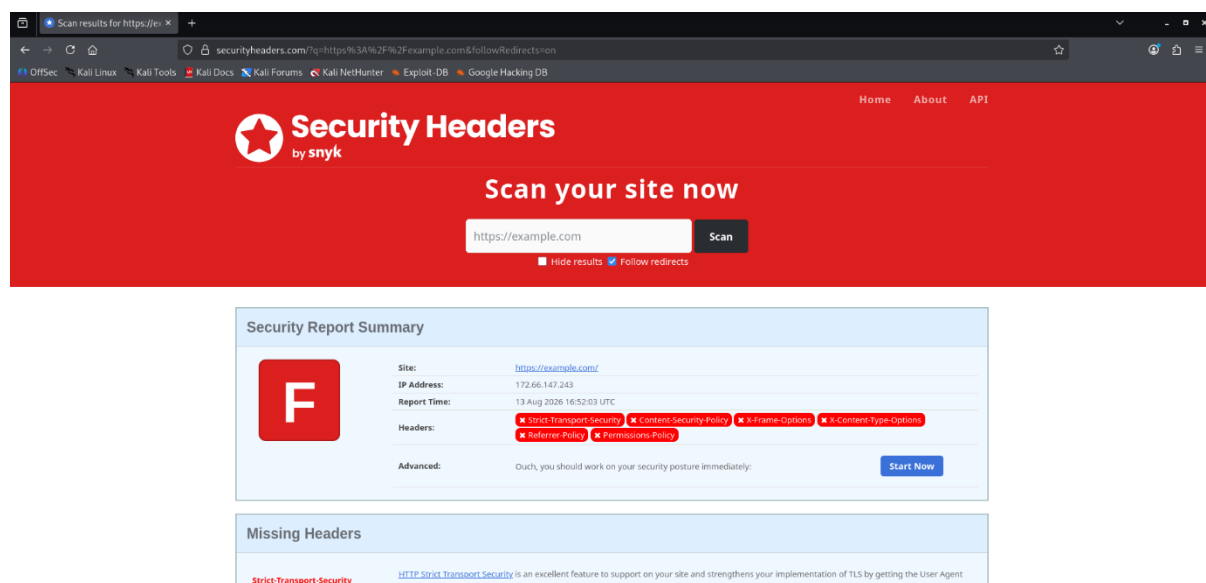


Figure 11: Security Headers analysis showing missing security headers

11.3 Apache Configuration

Security headers were configured for the local Apache server hosting DVWA.

The Apache configuration was checked using:

```
sudo apache2ctl configtest
```

The result was:

REPORT BY MANASVI RAJPUT

Syntax OK

Apache was then reloaded using:

```
sudo systemctl reload apache2
```

11.4 Header Verification

The configured headers were verified using:

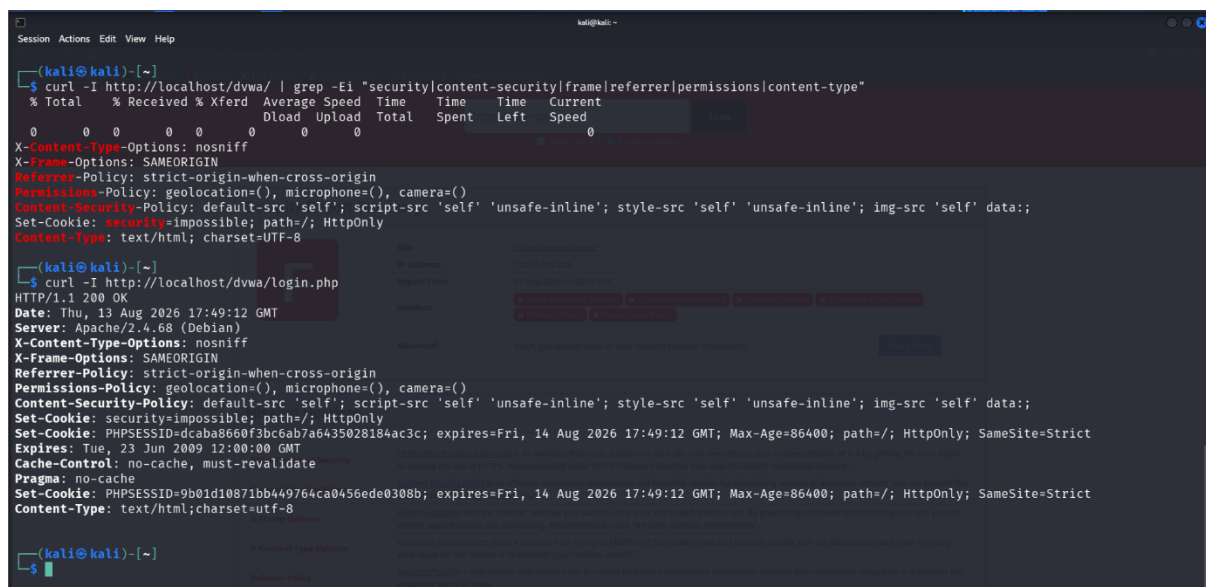
```
curl -I http://localhost/dvwa/login.php
```

The response included:

```
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), microphone=(), camera=()
Content-Security-Policy: default-src 'self'; ...
```

The server returned:

HTTP/1.1 200 OK



```
(kali@kali)-[~]
└─$ curl -I http://localhost/dvwa/login.php | grep -Ei "security|content-security|frame|referrer|permissions|content-type"
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
0         0     0    0         0      0         0      0
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), microphone=(), camera=()
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline'; img-src 'self' data;;
Set-Cookie: security=impossible; path=/; HttpOnly
Content-Type: text/html; charset=UTF-8

(kali@kali)-[~]
└─$ curl -I http://localhost/dvwa/login.php
HTTP/1.1 200 OK
Date: Thu, 13 Aug 2026 17:49:12 GMT
Server: Apache/2.4.68 (Debian)
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), microphone=(), camera=()
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline'; img-src 'self' data;;
Set-Cookie: security=impossible; path=/; HttpOnly
Set-Cookie: PHPSESSID=dcaba8660f3bc6ab7a6435028184ac3c; expires=Fri, 14 Aug 2026 17:49:12 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
Expires: Tue, 23 Jun 2009 12:00:00 GMT
Cache-Control: no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: PHPSESSID=9b01d10971bb449764ca0456ede0308b; expires=Fri, 14 Aug 2026 17:49:12 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
Content-Type: text/html; charset=utf-8
```

Figure 12: Verification of configured HTTP security headers using curl

11.5 Header Purpose

Header	Purpose
Content-Security-Policy	Restricts the sources from which browser content can be loaded and helps reduce XSS risk.
X-Frame-Options	Helps prevent unauthorized framing and clickjacking.

Header	Purpose
X-Content-Type-Options	Prevents MIME-type sniffing.
Referrer-Policy	Controls the amount of referrer information sent by the browser.
Permissions-Policy	Controls access to browser features such as camera, microphone and geolocation.
Strict-Transport-Security	Forces browsers to use HTTPS after the policy is received over a secure connection.

11.6 Result

The Apache server successfully returned the configured security headers, confirming that the header configuration was applied successfully.

12. Summary of Findings

Test	Result	Status
SQL Injection	Vulnerable functionality demonstrated in DVWA	Vulnerable
XSS	Vulnerable functionality demonstrated	Vulnerable
CSRF	Incorrect CSRF token rejected	Protection observed
LFI	/etc/passwd successfully accessed	Vulnerable
RFI	Remote inclusion not demonstrated; allow_url_include disabled	Mitigated by configuration
Burp Interception	Request successfully intercepted and modified	Completed
Burp Intruder	Multiple payloads tested and responses compared	Completed
Security Headers	Missing headers identified on test site	Improvement Required
Apache Headers	Security headers configured and verified	Completed

13. Overall Security Analysis

The practical testing demonstrated several important aspects of web application security.

The DVWA environment intentionally contains vulnerable functionality, allowing common vulnerabilities such as SQL Injection, XSS and LFI to be demonstrated safely in a controlled environment.

The LFI test successfully demonstrated the consequences of insufficient file-path validation, while the RFI test demonstrated the importance of secure PHP configuration. In particular, `allow_url_include` was disabled, preventing the intended remote PHP inclusion mechanism from being demonstrated.

Burp Suite demonstrated that HTTP requests should always be considered untrusted because parameters can be modified before reaching the server. Intruder further demonstrated how automated request generation can be used during authorized security testing to identify differences in application responses.

Finally, HTTP security headers were analyzed and configured at the Apache server level. Verification using `curl` confirmed that the configured headers were being returned by the DVWA server.

14. Recommended Mitigations

Based on the testing performed, the following security controls are recommended:

1. **SQL Injection**
 - Use parameterized queries/prepared statements.
 - Validate user input.
 - Use least-privileged database accounts.
2. **XSS**
 - Apply context-aware output encoding.
 - Validate input.
 - Implement an appropriate CSP.
3. **CSRF**
 - Use unpredictable CSRF tokens.
 - Validate tokens server-side.
 - Use appropriate `SameSite` cookie settings.
4. **LFI**
 - Use file allowlists.
 - Avoid direct user-controlled file paths.
 - Restrict filesystem permissions.
5. **RFI**
 - Keep `allow_url_include` disabled.
 - Avoid remote file inclusion.
 - Validate all file-related parameters.
6. **Authentication Attacks**
 - Implement rate limiting.
 - Use account lockout controls.
 - Apply MFA.
 - Monitor repeated authentication failures.
7. **HTTP Security**
 - Configure appropriate security headers.
 - Use HTTPS in production.
 - Implement a restrictive CSP.

- Apply secure cookie attributes.

15. Conclusion

Task 3 provided practical experience in identifying and analyzing common web application security vulnerabilities. Testing was performed against DVWA in a controlled local environment using manual browser testing, HTTP request analysis, Burp Suite and command-line tools.

The exercises demonstrated how vulnerabilities such as SQL Injection, XSS and LFI can arise from inadequate input validation and insecure application design. The CSRF exercise demonstrated the role of anti-CSRF tokens in preventing unauthorized state-changing requests.

Burp Suite provided practical experience in intercepting, modifying and automating HTTP requests. Intruder testing demonstrated how different payloads can be systematically tested and compared using response characteristics.

The file inclusion exercises also highlighted the importance of secure server configuration. While LFI was successfully demonstrated, RFI could not be demonstrated because `allow_url_include` was disabled in the PHP configuration.

Finally, security headers were analyzed and appropriate headers were configured in Apache. The configuration was successfully verified using `curl`, demonstrating how server-level security controls can improve the security posture of a web application.

Overall, the task provided practical understanding of **web vulnerability identification, exploitation in a controlled environment, HTTP traffic analysis, security configuration, and vulnerability mitigation.**